

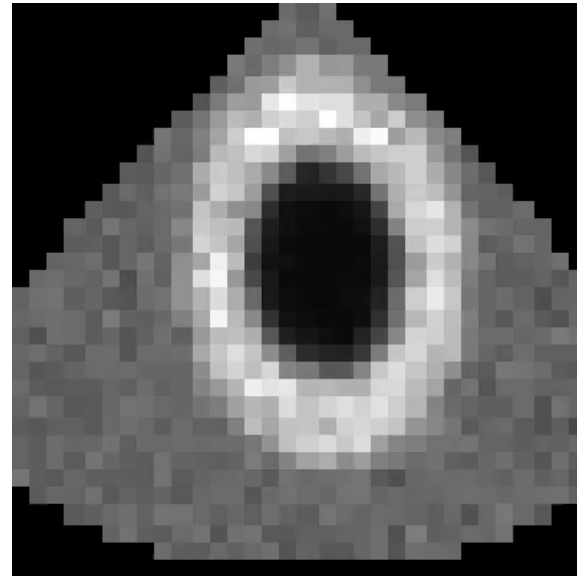
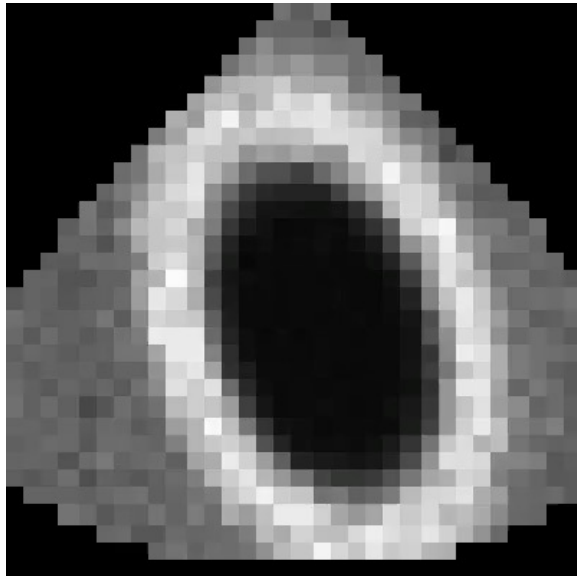
Adversarial Training

Marios S. Pattichis, FEAMBES, SM NAI

Department of Electrical and Computer Engineering

The University of New Mexico, USA

Simulated videos for EF=20 (left), EF=70 (right)



Ejection fraction (EF): the fraction of blood the left ventricle pumps out with each heartbeat.

$EF = (EDV - ESV) / EDV$ EDV, ESV: ventricle volume when most filled / most emptied

Normal: about 55–70%. 40% or less: reduced (heart failure).

In our videos (2D): $EF = 1 - \text{smallest cavity area} / \text{largest cavity area}$. A bad approx.

Adversarial attack over mask, random start, and steps

```
def pgd(net, x, y, eps, steps=10, alpha=None, mask=None, random_start=True):  
    """L-infinity PGD on videos. mask (same shape as x, 0/1) limits which pixels may change."""  
    alpha = alpha or 2.5 * eps / steps  
    mask = torch.ones_like(x) if mask is None else mask  
    delta = torch.empty_like(x).uniform_(-eps, eps) * mask if random_start else torch.zeros_like(x)  
    for _ in range(steps):  
        delta.requires_grad_(True)  
        loss = F.cross_entropy(net((x + delta).clamp(-1, 1)), y)  
        grad, = torch.autograd.grad(loss, delta)  
        delta = (delta.detach() + alpha * grad.sign() * mask).clamp(-eps, eps)  
    return (x + delta).clamp(-1, 1).detach()  
  
def fgsm(net, x, y, eps):  
    return pgd(net, x, y, eps, steps=1, alpha=eps, random_start=False)
```

Control the size of the attack using alpha.

multiple steps

delta = uniform random noise input.

Attack using the sign of the gradient of the added delta.

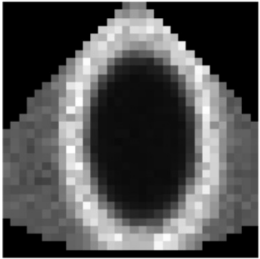
Original FGSM sets alpha=eps instead of optimizing better over 10 steps.

Standard model accuracy: clean 100.0% → FGSM 44.9% → PGD 6.0%

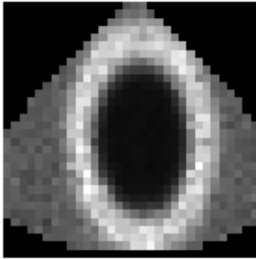
Original FGSM attack is weaker. Taking 10 gradient steps is more effective and better optimized.

An Example of an Adversarial Attack

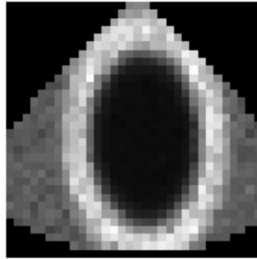
clean: reduced EF (100%)
frame 0



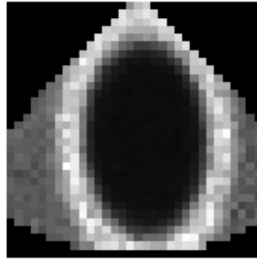
frame 4



frame 8

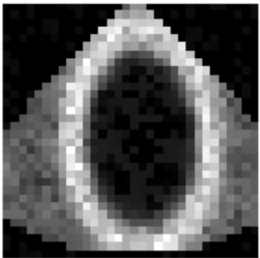


frame 12

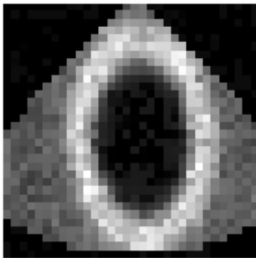


Reduced ejection fraction is correctly classified.

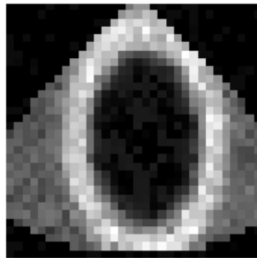
PGD: normal EF (100%)
frame 0



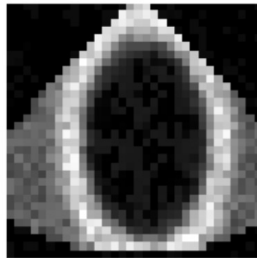
frame 4



frame 8

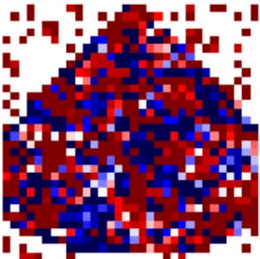


frame 12

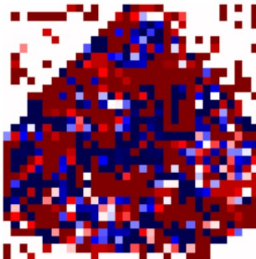


A small attack keeps the same look but now gets misclassified!

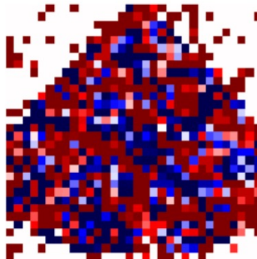
perturbation (max 0.1)
frame 0



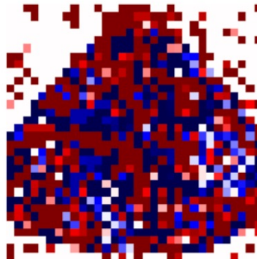
frame 4



frame 8



frame 12



Noise that was added to the original (=top video).

Adversarial Training

```
def train(adversarial=False, steps=800, batch=64, lr=1e-3):
    net = ViT3D().to(device)
    opt = torch.optim.AdamW(net.parameters(), lr=lr, weight_decay=0.05)
    sched = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=lr, total_steps=steps, pct_start=0.1)
    scaler = torch.amp.GradScaler(enabled=use_amp)
    t0 = time.time()
    for step in range(1, steps + 1):
        x, y, _ = make_echo(batch)
        if adversarial:
            eps = EPS * min(1.0, step / (0.4 * steps))
            x = fgsm_rs(net, x, y, eps)
        net.train()
        with torch.autocast(device_type=device, dtype=torch.float16, enabled=use_amp):
            loss = F.cross_entropy(net(x), y)
        opt.zero_grad(set_to_none=True)
        scaler.scale(loss).backward()
        scaler.step(opt); scaler.update(); sched.step()
        if step % 200 == 0:
            print(f"step {step:4d}    loss {loss.item():.3f}    {time.time() - t0:5.0f} s")
    return net.eval()

@torch.no_grad()
def accuracy(net, x=None, y=None):
    x, y = (x_test, y_test) if x is None else (x, y)
    return torch.cat([net(x[i:i + 500]).argmax(1) for i in range(0, len(x), 500)]).eq(y).float().mean().item()
```

Achieve fast convergence by training scheduling the learning rate.

Prevent zero-gradient components through scaling.

Linearly increase eps and then keep at 1.0.

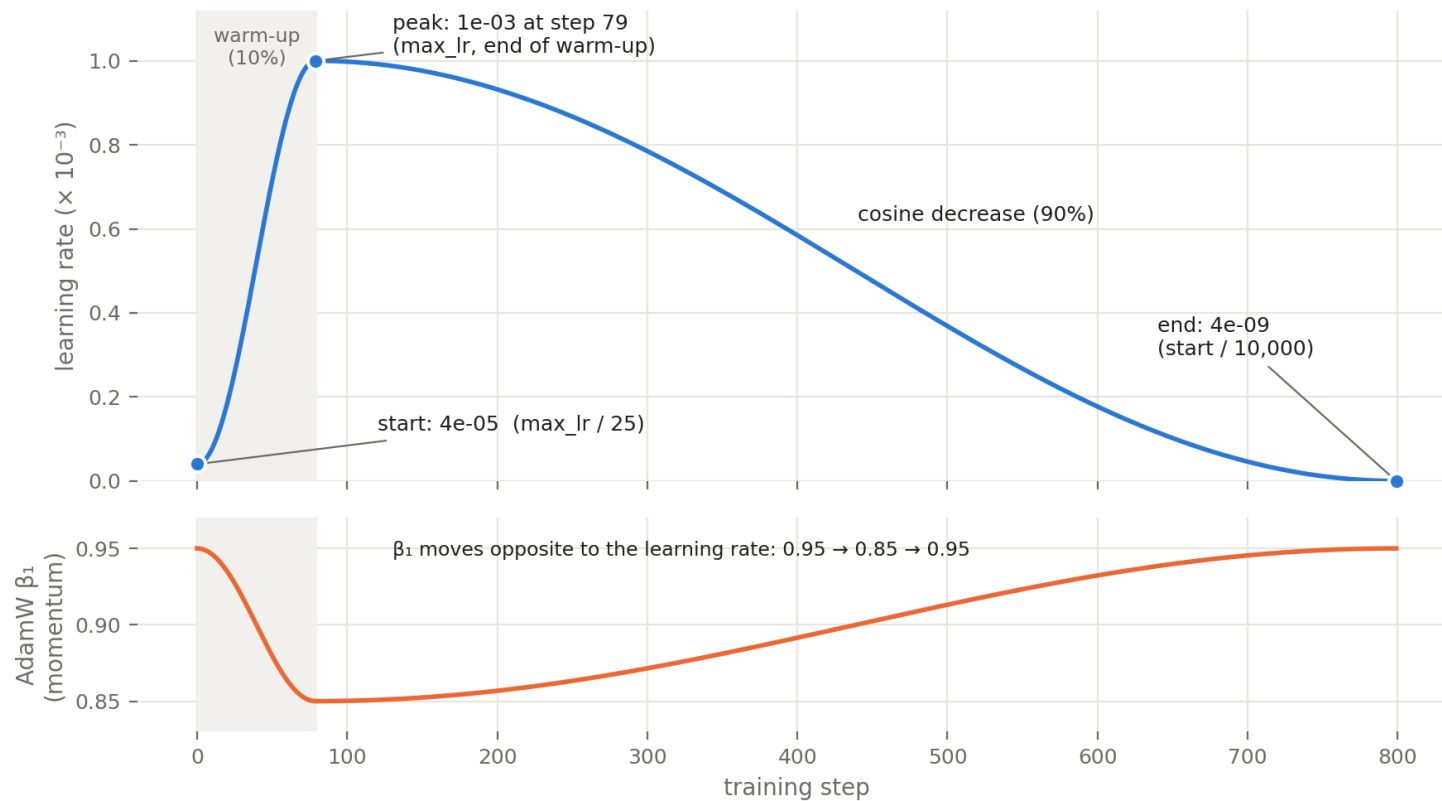
Attack x but keep the label y the same.

epsilon warm-up: 0 -> EPS over the first 40%

attack the batch first (section 6)

Fast training using learning rate and momentum scheduling

OneCycleLR schedule used in the notebooks ($\text{max_lr} = 1\text{e-}3$, 800 steps, $\text{pct_start} = 0.1$)



Improved results for reasonable attacks

